Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Tek boyutlu faks kodlamasının tekrarı
  - Faks makinesi belgeyi, içinde ne olursa olsun, bir görüntü olarak gönderir. Görüntü piksellerden değil, **pels** (picture element) adı verilen siyah-beyaz noktalardan oluşur.
  - Run uzunlukları birçok dokümanın istatistiğiyle kodlanmıştır: 0-63 arası için **termination**, 64'ün katları için **make-up** kodları vardır; belirli bir değerden sonra siyah ve beyaz make-up kodları aynıdır (ekrandan, 00:20:00). Örneğin 76 beyaz = 64 beyaz (make-up) + 12 beyaz (termination).
  - Her satır ayrı kodlanır, sonuna EOL konur, her satırın başına bir beyaz pels eklenir. Satır sayısı kadar EOL yoksa belge yeniden istenebilir; satır içindeki bir hata yalnız o satırı bozar. Bu yönteme one-dimensional coding (MH, Modified Huffman) denir.
  - Gray scale ya da renkli bir belgede run'lar kısalır (örneğin arka arkaya 832 beyaz yerine 5 beyaz, 4 siyah, 5 beyaz …) ve tek boyutlu kodlamanın başarısı düşer.
- İki boyutlu faks kodlaması (ekrandan, 00:27:00)
  - Tek boyutlu kodlama gray scale resimlerde iyi sonuç vermediği için geliştirilmiştir. Grup 3'lü kodlamada faks makinelerinde seçimlik olarak bulunur.
  - Her EOL kodundan sonra fazladan bir bit kullanılır: 1 ise gelecek satır 1D, 0 ise 2D kodlanır.
  - Slaytta 2D kodlama **MMR (Modified Modified READ)** olarak adlandırılır; READ = Relative Element Address Designate.
  - Kodlanan satıra **coding line**, bir üstündeki daha önce kodlanmış satıra **reference line** denir. İki satır karşılaştırılır ve aralarındaki farklar kodlanır. Örneğin üst satır tamamen beyaz, alttaki satırda tek bir siyah pels varsa yalnız bu fark kodlanır.
  - Sayfanın ilk satırının üstünde referans satırı olmadığı için ilk satır her zaman 1D kodlanır.
- 2D kodlamada güvenlik ve k parametresi (ekrandan, 00:30:00)
  - 2D kodlama 1D'den daha az güvenilirdir: bir hata oluştuğunda nereden geldiği bilinemez ve hata belgenin sonuna kadar sürer. Bu yüzden belli aralıklarla 2D kodlama kesilir ve bir satır 1D kodlanır.
  - T4 (Grup 3): bir satır 1D kodlandıktan sonra en fazla $k-1$ satır 2D kodlanır. Standart çözünürlükte $k = 2$, yüksek çözünürlükte $k = 4$ (bir satır 1D, üç satır 2D, sonra yine bir satır 1D …).
  - T6 (Grup 4) kodlamada böyle bir zorunluluk yoktur; bütün satırlar 2D gönderilebilir, ama hata olursa karşı taraf belgeyi hatalı alır.
- Parametreler ve başlangıç durumu (ekrandan, 00:46:35)
  - Coding line'da a_0, a_1, a_2; reference line'da b_1, b_2 vardır. Bunlar pointer gibi düşünülebilir.
  - Tarama başladığında a_0 coding line'daki ilk başlangıç beyaz pels'i (1D'de her satırın başına eklenen beyaz pels), a_1 ilk siyah pels'in başlangıcını, a_2 ondan sonraki ilk beyaz pels'in başlangıcını gösterir. b_1 ve b_2 reference line'daki birinci ve ikinci run'ın başlangıcını gösterir.
  - Beş parametrenin konumuna göre üç mod vardır: pass, vertical ve horizontal.
- Pass mode (ekrandan, 00:33:15)
  - Koşul: b_2, a_1'in sol tarafındadır; a_1 ile b_1 aynı hizada olamaz.
  - Slayta göre çıkışa `0001` + ($b_1 b_2$) uzunluğunun 1D tablodaki kodu yazılır.
  - Sonra a_0, b_2'nin altına taşınır; b_1 ve b_2 güncellenir; a_1 ve a_2 değişmez.

- Vertical mode (ekrandan, 00:40:00)
  - Koşul: a_1 ile b_1 arasında sağa veya sola doğru en fazla 3 pels vardır. Yedi durum için yedi kod üretilir: `V(0) = 1`, `VR(1) = 011`, `VR(2) = 000011`, `VR(3) = 0000011`, `VL(1) = 010`, `VL(2) = 000010`, `VL(3) = 0000010`. VR, a_1'in b_1'in sağında; VL, solunda olduğu durumdur.
  - Sonra a_0 a_1'in yerine, a_1 a_2'nin yerine geçer, a_2 bir sonraki run'ın başına gider; b_1 b_2'nin yerine geçer, b_2 bir sonraki run'ın başına gider.
- Horizontal mode (ekrandan, 00:42:20)
  - Koşul: a_1 ile b_1 arasında sağa veya sola doğru 3'ten fazla pels vardır.
  - Çıkışa `001` + a_0a_1 (beyaz) run'ının kodu + a_1a_2 (siyah) run'ının kodu yazılır; kodlar 1D (MH) termination tablosundan alınır. Slayttaki örnekte 3 beyaz ve 7 siyah vardır.
  - Sonra a_0, a_2'nin yerine geçer; a_1, a_2, b_1, b_2 güncellenir.
- Extension kodu (ekrandan, 00:46:35)
  - `0000001000`: sayfanın sonuna gelmeden kodlama işlemini iptal etmek veya sayfanın geri kalanını farklı kodla ya da sıkıştırmadan (ASCII karşılıklarıyla) iletmek için kullanılır.
- 2D kodlama örneği, "Short example" (ekrandan, 00:50:00 ve 01:00:00)
  - Adımlar: VL(1) → `010`; V(0) → `1`; horizontal, 3 beyaz + 4 siyah → `001 1000 011`; pass + 2 beyaz → `0001 0111`; VR(2) → `000011`; VL(2) → `000010`; horizontal, 4 beyaz + 7 siyah → `001 1011 00011`.
  - Satırın tam çıkışı: `010 1 001 1000 011 0001 0111 000011 000010 001 1011 00011`, sonra EOL.
  - Horizontal modda sayılan pels'ler reference line'dan değil, coding line'dan sayılır.
  - Bir sonraki satırda bu satır reference line olur; $k-1$ satır 2D kodlandıktan sonra bir satır 1D kodlanır.
- Faks sıkıştırması da istatistiksel bir yöntemdir
  - Siyah ve beyaz run'ların sayıları çıkarılır, olasılığı yüksek olana az, düşük olana çok bit verilir. Ama önceden çıkarılmış bir tablo kullanıldığı için statik bir kodlamadır.
- Aritmetik kodlamaya giriş (ekrandan, 01:20:00)
  - Shannon-Fano yalnız olasılıklar ikinin negatif kuvvetleriyse iyi sonuç veriyordu; Huffman ondan daha iyi sonuç veriyordu.
  - Olasılığı $0{,}4$ olan bir sembole atanan bit $-\log_2 0{,}4 \approx 1{,}32$'dir; ama tek bir sembole 1 ya da 2 bit verilebilir.
  - Aritmetik kodlamada tek bir sembole değil, bütün input stream'e (bir kelime, satır, sayfa, kitap) tek bir kod atanır. Kod $[0, 1)$ aralığında bir sayıdır; sembol sayısı arttıkça aralık daralır ve sayının dijit sayısı artar.
  - Kod «9746509» ise «0,9746509» olarak yorumlanır; baştaki «0» output stream'e yazılmaz.
  - İki pass'te çalışır: ilk pass'te sembollerin kullanım sıklıkları (olasılıkları) çıkarılır, ikinci pass'te sıkıştırma yapılır.
- Aralıkların atanması ve formüller (ekrandan, 01:30:00 ve 01:40:00)
  - Semboller olasılıklarına göre $[0, 1)$ aralığına art arda dağıtılır: $a_1 = 0{,}4$, $a_2 = 0{,}5$, $a_3 = 0{,}1$ için $a_1 \to [0; 0{,}4)$, $a_2 \to [0{,}4; 0{,}9)$, $a_3 \to [0{,}9; 1)$. Sıra ters de seçilebilir; sonuç değişmez.
  - Range $=$ OldHigh $-$ OldLow
  - NewHigh $=$ OldLow $+$ Range $\times$ HighRange(x)
  - NewLow $=$ OldLow $+$ Range $\times$ LowRange(x)
  - LowRange(x) ve HighRange(x), x sembolünün alt ve üst sınırlarıdır.
- Örnek: «a2a2a2a3» (ekrandan, 01:33:40)
  - a_2: Low $= 0 + (1 - 0) \times 0{,}4 = 0{,}4$; High $= 0 + (1 - 0) \times 0{,}9 = 0{,}9$.
  - a_2: Low $= 0{,}4 + 0{,}5 \times 0{,}4 = 0{,}6$; High $= 0{,}4 + 0{,}5 \times 0{,}9 = 0{,}85$.
  - a_2: Low $= 0{,}6 + 0{,}25 \times 0{,}4 = 0{,}7$; High $= 0{,}6 + 0{,}25 \times 0{,}9 = 0{,}825$.
  - a_3: Low $= 0{,}7 + 0{,}125 \times 0{,}9 = 0{,}8125$; High $= 0{,}7 + 0{,}125 \times 1 = 0{,}825$.
  - Çıkış $[0{,}8125; 0{,}825)$ aralığında bir sayıdır. Bazıları low değerini, bazıları aradaki bir değeri gönderir. 8125 sayısı 13 bitle yazılır; 4 sembol ASCII ile 32 bit tutardı.
- Kurallar ve ortalama kod uzunluğu (ekrandan, 01:37:20)
  - (1) $[0, 1)$ ile başlanır. (2) Her sembol için geçerli aralık olasılıklarla orantılı alt aralıklara bölünür ve sembolün alt aralığı yeni geçerli aralık olur. (3) Bütün input işlenince çıkış, geçerli aralıktaki herhangi bir sayıdır.

  - Her sembol aralığı küçültür; bu da kodun daha fazla bitle ifade edilmesine yol açar.
  - Ortalama kod uzunluğu = output bit uzunluğu / input stream'deki sembol sayısı. «a2a2a2a3» için $13/4 = 3{,}25$ bit.
  - Açılabilmesi için header'da semboller ve sıklıkları da gönderilir; alıcı aralık tablosunu bunlardan yeniden kurar.
- «SWISS MISS» örneği (ekrandan, 01:40:00)
  - Tablo: S 5 → $[0{,}5; 1{,}0)$; W 1 → $[0{,}4; 0{,}5)$; I 2 → $[0{,}2; 0{,}4)$; M 1 → $[0{,}1; 0{,}2)$; boşluk 1 → $[0{,}0; 0{,}1)$. Toplam kümülatif frekans 10; CumFreq sütunu (5, 4, 2, 1, 0) kod çözücü tarafından kullanılır.
  - Kodlama sonucunda aralık $[0{,}71753375; 0{,}717535)$ olur (python ile doğrulandı).
- Kod çözme (ekrandan, 01:50:00)
  - Kod çözücü karşı yönde çalışır ve tabloyu header'dan yeniden kurar. İlk dijit olarak 7'yi okur ve bunun 0,7 olduğunu bilir.
  - Her adımda: sayının düştüğü aralığın sembolü yazılır, sayıdan sembolün low değeri çıkarılır, sonuç sembolün range'ine bölünür.
  - $0{,}71753375$ → S ($0{,}4350675$) → W ($0{,}350675$) → I ($0{,}753375$) → S ($0{,}50675$) → S ($0{,}0135$) → boşluk ($0{,}135$) → M ($0{,}35$) → I ($0{,}75$) → S ($0{,}5$) → S (0). Sonuç 0 olunca işlem biter: «SWISS MISS».
- Sonu çözülemeyen girdi ve EOF sembolü (ekrandan, 01:55:30 ve 02:00:00)
  - Aralıklar $a_3 \to [0{,}0; 0{,}023162)$, $a_2 \to [0{,}023162; 0{,}998162)$, $a_1 \to [0{,}998162; 1{,}0)$ olsun. «a2a2a1a3a3» kodlanınca low = $0{,}99462270125$ olur.
  - Çözümde a_2, a_2, a_1 bulunur; a_1'den sonra değer 0 olur ve algoritma durur. Son iki a_3 çözülmez, çünkü a_3'ün aralığı 0'dan başlar.
  - Çözüm: ek bir **EOF** sembolü kullanmak (çok küçük ama sıfır olmayan bir olasılıkla, örneğin $[0{,}999999; 1{,}0)$) ya da kodlayıcının output stream'e önce input stream'in boyutunu yazması. «a3a3a3a3eof» örneğinde dört a_3 ve EOF doğru çözülür.
- Integer ile aritmetik kodlama (ekrandan, 02:10:00)
  - Float sayılarla çalışmak işlemi uzatır; bir dosyanın kodu çok büyük bir sayı olabilir. Bu yüzden High ve Low 4 dijitlik integer değişkenlerde tutulur.
  - Low ve High'ın en soldaki dijitleri aynıysa o dijit output stream'e yazılır ve değişkenler sola ötelenir; sağdan Low'a 0, High'a 9 eklenir. Ölçeklendirmede değerler 10.000 ile çarpılır, High'dan 1 çıkarılır.
  - «SWISS MISS» için output stream'e «717533750» gönderilir (python ile doğrulandı).
- Integer kod çözme (ekrandan, 02:23:40)
  - index = $\frac{(\text{Code}-\text{Low}+1)\times 10-1}{\text{High}-\text{Low}+1}$
  - Low = Low + $(\text{High} - \text{Low} + 1) \times \text{LowCumFreq}[X]/10$
  - High = Low + $(\text{High} - \text{Low} + 1) \times \text{HighCumFreq}[X]/10 - 1$
  - Buradaki 10, toplam kümülatif frekanstır; örneğe özeldir.
  - Low = 0000, High = 9999, Code = 7175 ile başlanır. index = $7{,}1759$ → S; index = $4{,}3518$ → W, 7 ötelenir, Code = 1753; index = $3{,}5078$ → I, 1 ötelenir, Code = 7533; ...
- Sonuç ve karşılaştırma (ekrandan, 02:30:00)
  - Pratikte bütün işlemler binary sayılar üzerinde yapılır. «SWISS MISS» için Low ve High'ın binary karşılıkları belirli bir yere kadar aynıdır; 10 sembol 29 bitle (yaklaşık 4 byte) sıkıştırılır. Her sembol 8 bitle kodlansaydı 80 bit gerekirdi.
  - Aritmetik kodlama genelde Huffman'dan biraz daha iyi sonuç verir; ama Huffman'ı kodlamak daha kolaydır. Açık kaynak kodlarda çoğunlukla float versiyon görülür.
- PPM'e giriş (ekrandan, 02:38:05 ve 02:40:00)
  - PPM, Cleary ve Witten tarafından ortaya atılmış, Moffat tarafından iyileştirilmiştir. Order-N Markov modeline dayanır: bir sembole olasılık verirken o sembolden önce gelen N sembole bakılır.
  - Kodlayıcı ve kod çözücü metnin ilerisini göremez, bu yüzden geçmişteki içerikle ilgilenir. Model önce N geriye bakar; bulamazsa N-1, … 1'e kadar iner. Teoride N büyük olabilir, pratikte 3-4'ü geçen uygulama pek yoktur.
  - **İstatistiksel model:** 1217 sembolün 34'ü q ise olasılık $34/1217$'dir; yeni bir q ile $35/1217$ olur.
  - **İçerik (context) tabanlı model:** bir sembolün olasılığı, sembolün genel sıklığından bağımsız olarak context içindeki sıklığına göre verilir. Örneğin h %5, t %30 iken th'nin olasılığı çok daha fazladır.

  - **Statik içerik tabanlı:** alfabedeki bütün di-gram ve tri-gram olasılıklarından statik bir tablo kurulur; S okunduğunda önceki C context'ine göre olasılık tablodan çekilir.
  - **Adaptif:** olasılıklar anlık değişir. «A B A ...» girişinde önce $P(A) = 1$, B gelince $1/2$ ve $1/2$, yeni A ile $2/3$ ve $1/3$ olur. Yavaş ve karmaşık ama sıkıştırma oranı daha yüksektir.

Hocanın Özellikle Vurguladığı Kısımlar

- 2D kodlamada hata
  - 2D kodlamada hata bulunamaz ve bütün belge hatalı gidebilir. Bu yüzden $k-1$ satır 2D'den sonra bir satır 1D kodlanır; 1D satırın sonundaki EOL hatanın yayılmasını durdurur.
- Modların kodları ezberlenmez
  - Hoca modların kurallarını ve kodlarını ezbere bilmediğini, elindeki tablodan baktığını söyledi. Önemli olan pointer'ların konumuna bakarak modu bulmak ve pointer'ları doğru güncellemektir; tablo verilip "bunu çöz" denebilir.
- Coding line'dan say
  - Horizontal modda a_1 ile b_1 arasındaki uzaklığa bakılır; ama yazılan run'lar (a_0a_1, a_1a_2) coding line'dan sayılır, reference line'dan değil.
- Aritmetik kodlamanın farkı
  - Karakterlere değil, bütün input stream'e tek bir kod verilir. Her şey iki formül üzerine kuruludur: her yeni sembol aralığı daraltır.
- Aralığı 0'dan başlayan son sembol
  - Girdinin son sembolleri aralığı 0'dan başlayan sembolse çözücü 0'da durur ve gerisini çözemez. Bunun için EOF sembolü eklenir.
- Temel yöntemler
  - Bu ders farklı üniversitelerde farklı algoritmalarla anlatılsa da Huffman, aritmetik kodlama, LZ ve PPM her yerde ortak olarak verilir.

Kısa Tekrar Notları

- 1D = MH; 2D = slaytta MMR (READ: Relative Element Address Designate).
- EOL'dan sonraki bit: 1 → sonraki satır 1D, 0 → 2D.
- T4 (Grup 3): 1 satır 1D + en fazla $k-1$ satır 2D; standart $k = 2$, yüksek $k = 4$. T6 (Grup 4): zorunluluk yok.
- Pass: b_2, a_1'in solunda → `0001` (+ b_1b_2); $a_0 \to b_2$'nin altı; a_1, a_2 değişmez.
- Vertical: $|a_1b_1| \le 3$ → `1`, `011`, `000011`, `0000011` (VR), `010`, `000010`, `0000010` (VL).
- Horizontal: $|a_1b_1| > 3$ → `001` + $M(a_0a_1)$ + $M(a_1a_2)$; $a_0 \to a_2$.
- Extension: `0000001000`.
- Short example: `010 1 001 1000 011 0001 0111 000011 000010 001 1011 00011`.
- Aritmetik kodlama: bütün girdiye $[0, 1)$ aralığında tek sayı; iki pass.
- NewLow = OldLow + Range × LowRange(x); NewHigh = OldLow + Range × HighRange(x).
- «a2a2a2a3» → $[0{,}8125; 0{,}825)$.
- «SWISS MISS» → $[0{,}71753375; 0{,}717535)$; integer versiyon çıkışı «717533750»; 29 bit.
- Çözme: sembolü bul, low'u çıkar, range'e böl; 0 olunca dur.
- Aralığı 0'dan başlayan son sembol → EOF ekle ya da girdi boyutunu yaz.
- Integer: 4 dijit; soldaki dijitler aynıysa yaz ve ötele (Low'a 0, High'a 9).
- PPM: order-N Markov, context tabanlı; N geriye bak, bulamazsan N-1 ...

Detaylı Açıklamalar

- **İki boyutlu kodlamanın fikri.** A4 bir belgenin peş peşe gelen iki satırı çoğu zaman birbirine çok benzer. Üst satır tamamen beyaz ve alt satır da beyazsa fark yoktur; alt satır üsttekinin aynısı gibi kodlanabilir. Üst satır boş, alttakinde tek bir siyah pels varsa yalnız bu fark kodlanır. Alt satırın tamamını kodlamak yerine farkı kodlamak daha iyi sıkıştırma verir. Hoca, 2D kodlamanın anlatmasının zahmetli olduğunu, ama kodlandıktan sonra çok hızlı ve iyi bir performansla çalıştığını söyledi.

- **Faksta "satır".** Faks için satır, belgede gözle görülen yazı satırı değildir; bir yazı satırının yüksekliği belki 10 scan line'a karşılık gelir. İnce bir scan line'ın bozuk gitmesi gözle kolayca tolere edilir.
- **Modu bulma sırası.** Hoca örnekte her adımda sırayla kontrol etti: önce pass mode mu (b_2, a_1'in solunda mı?), değilse vertical mode mu (a_1 ile b_1 arası en fazla 3 pels mi?), o da değilse horizontal mode. Mod bulunur, kod yazılır, pointer'lar güncellenir ve satırın sonuna kadar bu tekrarlanır.
  - Ders sonrası not: Vertical mode slaytında koşul "2'den fazla pels olmamalıdır" diye yazar (ekrandan, 00:40:00); ama aynı slayttaki tablo ± 3'e kadar gider ve horizontal mode koşulu "3'ten fazla" olduğundan doğru koşul $|a_1 b_1| \leq 3$'tür. Hoca da 00:40:12'de "3'ten fazla olamaz" dedi.
  - Ders sonrası not: Slayt ve örnek, pass mode kodundan sonra $b_1 b_2$ uzunluğunun kodunu yazar (`0001 0111`). ITU T.4 standardında pass mode kodu büyük olasılıkla yalnız `0001`'dir ve arkasından run uzunluğu yazılmaz.
  - Ders sonrası not: Slaytta 2D kodlama MMR olarak geçer. Büyük olasılıkla Grup 3'teki (T.4) 2D kodlamanın adı MR (Modified READ), MMR (Modified Modified READ) ise Grup 4'ün (T.6) kodlamasıdır.
- **Short example'ın adımları.** Pointer'lar satırın başında konur. (1) b_2, a_1'in solunda değil, a_1 ile b_1 arası 3'ten az ve a_1 b_1'in solunda: VL(1) → `010`. (2) a_1 ile b_1 aynı hizada: V(0) → `1`. (3) a_1 ile b_1 arası 3'ten fazla: horizontal, coding line'da 3 beyaz ve 4 siyah → `001 + 1000 + 011`. (4) b_2, a_1'in solunda: pass, aradaki 2 beyaz → `0001 + 0111`. (5) a_1 b_1'in 2 sağında: VR(2) → `000011`. (6) VL(2) → `000010`. (7) horizontal, 4 beyaz ve 7 siyah → `001 + 1011 + 00011`. Termination kodları slayttaki tabloyla karşılaştırıldı ve tuttu (3 beyaz `1000`, 4 siyah `011`, 2 beyaz `0111`, 4 beyaz `1011`, 7 siyah `00011`).
- **Aritmetik kodlamada sayı küçülür.** Aralık her sembolle daralır. Sıfır ile bir arasında sonsuz sayı olduğu için girdi uzadıkça aralık sıfıra yakın bir genişliğe iner ve sayının dijit sayısı artar. Hoca, bir kitabın tamamını tek sayıyla kodlamanın çok zor ve zaman alan bir süreç olacağını söyledi.
- **Bit sayısı.** «a2a2a2a3» için 8125 sayısı yazılır. $2^{13} = 8192 > 8125$ olduğundan 13 bit yeterlidir (python ile kontrol edildi); ortalama kod uzunluğu $13/4 = 3{,}25$ bit/sembol olur.
  - Ders sonrası not: Hoca 01:35:14'te bitleri sayarken 14 bit dedi, 01:37:07'de 13 bit diye düzeltti; doğrusu 13 bittir.
- **Tablonun kuruluşu.** Kitaptaki «SWISS MISS» tablosu aralıkları 1'den başlayıp olasılıkları çıkararak kurar (S $[0{,}5; 1)$, W $[0{,}4; 0{,}5)$, ...). Hoca 0'dan başlayıp olasılıkları ekleyerek kurmayı daha kolay bulduğunu söyledi; aralıklar aynı çıkar. Kümülatif frekans: boşluk 0 → 1, M 1 → 2, I 2 → 4, W 4 → 5, S 5 → 10.
- **«SWISS MISS» kodlama adımları (python ile doğrulandı).** S $[0{,}5; 1)$; W $[0{,}7; 0{,}75)$; I $[0{,}71; 0{,}72)$; S $[0{,}715; 0{,}72)$; S $[0{,}7175; 0{,}72)$; boşluk $[0{,}7175; 0{,}71775)$; M $[0{,}717525; 0{,}71755)$; I $[0{,}71753; 0{,}717535)$; S $[0{,}7175325; 0{,}717535)$; S $[0{,}71753375; 0{,}717535)$.
- **Neden 0'da durulur?** «a2a2a1a3a3» örneğinde a_1 çözüldükten sonra $0{,}998162 - 0{,}998162 = 0$ olur. Devam edilse 0 her seferinde a_3'ün aralığına düşer ve a_3, a_3 çıkar; ama algoritmanın durma noktası 0'dır. Girdi a_3 ile başlasaydı ilk karakterden sonra çözme biterdi.
  - Ders sonrası not: Hoca 01:56:53'te sorunun, son karakterin "olasılık değeri sıfır" olmasından kaynaklandığını söyledi. Slayttaki örneğe göre a_3'ün olasılığı $0{,}023162$'dir; sorun, sembolün **aralığının 0'dan başlamasıdır** (low sınırı 0).
- **Integer versiyonun tablosu (ekrandan, 02:10:00).** Kolonlar: (1) okunan sembol, (2) yeni Low ve High, (3) ölçeklendirilmiş hâl, (4) output stream'e gönderilen dijit, (5) ötelemeden sonraki Low ve High. W adımında 7000 ve 7499'un ilk dijitleri aynıdır: 7 yazılır, 0000 ve 4999 olur. I adımında 1000 ve 1999 → 1 yazılır, 0000 ve 9999. Boşluk adımında 7500 ve 7749 → 7 yazılır, 5000 ve 7499. M adımında 5250 ve 5499 → 5, I adımında 3000 ve 3499 → 3. Sonda Low'daki 3750 de yazılır: «7175» + «3750» → «717533750». Ölçeklendirmede 9999, aslında 10.000 yani 1 gibi düşünülür (High'dan 1 çıkarıldığı için).
- **Integer çözmenin geri kalanı.** Slaytta ilk beş adım gösterildi; sonraki adımlar aynı formülle python ile hesaplandı (ekranda gösterilmedi): Code $= 7533$ ile index $0{,}1356 \rightarrow$ boşluk (7 ötelenir, Code $= 5337$); index $1{,}3516 \rightarrow$ M (5 ötelenir, Code $= 3375$); index $3{,}5036 \rightarrow$ I (3 ötelenir, Code $= 3750$); index $7{,}5018 \rightarrow$ S; index $5{,}0036 \rightarrow$ S.
- **Context'in uzunluğu.** "N tane geriye bak" riskli bir ifadedir: N = 10.000 seçilirse ilk 10.000 karakterin önünde o kadar geçmiş yoktur ve bunlar olduğu gibi kodlanır. Bu yüz-

den teoride N büyük olabilir ama pratikte küçük tutulur.

- **İkili kombinasyonlar.** A, B, C alfabesi için ikili kombinasyonlar (AB, BA, AC, CA, BC, CB, ...) önceden çıkarılır; metinde varsa kullanılır, yoksa kullanılmaz. İngilizcede T ve H tek başına da geçer, ama birlikte geçtikleri kelime sayısı fazladır; bu yüzden TH ikilisinin olasılığı farklıdır.
  - Ders sonrası not: Slaytta yeni bir q ile olasılığın $35/1217$ olacağı yazar. Yeni sembolle toplam sembol sayısı da bir artacağı için büyük olasılıkla doğrusu $35/1218$'dir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.